

班 级 2003012

学 号 许喆

西安电子科技大学

本科毕业设计论文



题 目 基于鸿蒙软总线的端侧集群动态管

理设计实现

学 院 计算机科学与技术学院

专 业 计算机科学与技术

学生姓名 许喆

导师姓名 杨鹏飞 教授

毕业设计（论文）诚信声明书

本人声明：本人所提交的毕业论文《基于鸿蒙软总线的端侧集群动态管理设计实现》是本人在指导教师指导下独立研究、写作的成果，论文中所引用他人的无论以何种方式发布的文字、研究成果，均在论文中加以说明；有关教师、同学和其他人员对本文的写作、修订提出过并为我在论文中加以采纳的意见、建议，均已在我的致谢辞中加以说明并深致谢意。

本论文和资料若有不实之处，本人承担一切相关责任。

论文作者：_____（签字） 时间： 年 月 日

指导教师已阅：_____（签字） 时间： 年 月 日

摘 要

鸿蒙操作系统是由华为公司开发的一款面向全场景的分布式操作系统。鸿蒙分布式软总线提供不区分通信方式的设备发现、连接、组网和传输功能，是鸿蒙操作系统的通信基座。然而，现有的分布式计算框架依赖 TCP/IP 协议下的远程过程调用（Remote Procedure Call, RPC）方式进行通信，但在多设备协同、多种通信机制的新型应用环境下存在局限性。本研究针对上述问题，对分布式软总线底层机制进行了深入研究，利用基于分布式软总线的分布式文件系统（Harmony Distributed File System, hmdfs），提出以分布式软总线为通信基座的端侧动态集群管理系统，完成多文本分布式推理，具体工作如下：

(1) 针对现有计算框架适配通信机制单一的问题，本文深入研究了分布式软总线的底层机制，设计了基于分布式软总线的去中心化端侧集群，具备命令答复，动态感知节点上下线的功能。并且设计了基于分布式软总线传输特点的设备信息传输机制，借助 cJSON 的序列化和反序列化过程，实现了组网设备资源特征收集。

(2) 针对 hmdfs 的结构特点，本文提出了一种分布式推理并行化机制，根据各个节点当前的计算资源特征，进行任务权重分配。借助 hmdfs 的全局一致的访问视图，非用户节点自取文本文件推理，保证了节点推理的异步性。

最后，本文对分布式软总线的 *SendBytes* 函数进行传输速率实验，经过六次独立重复实验后，本文采用最小二乘法对实验结果进行拟合，最终得出分布式软总线的传输速率为 8.9195MBytes/s，比较 TCP/IP 协议的 7.9MBytes/s，速率相对更快，信道利用率有所提升。之后对 hmdfs 进行了数据并行化速度测试，最终得知远端推理仅有 4.09% 的速度损失，因此本文提出的方法可以完成数据并行化速度提升。

关键词： 鸿蒙分布式软总线 OpenEuler embedded 操作系统 hmdfs 分布式推理

ABSTRACT

HarmonyOS is a distributed operating system developed by Huawei for all scenarios. The Harmony distributed softbus provides device discovery, connection, networking, and transmission functions that do not distinguish between communication methods, and is the communication base of the HarmonyOS. However, existing distributed computing frameworks rely on the RPC(Remote Procedure Call) method of communication under the TCP/IP protocol, but have limitations in new application environments with multi-device collaboration and multiple communication mechanisms. This research addresses the above problems, conducts an in-depth study on the underlying mechanism of distributed soft bus, and proposes an end-side dynamic cluster management system with distributed soft bus as the communication pedestal to complete the multi-text distributed reasoning by using the distributed file system (Harmony Distributed File System, hmdfs) based on distributed soft bus as the specific work is as follows:

(1) Aiming at the problem of a single communication mechanism adapted to existing computing frameworks, this paper deeply researches the underlying mechanism of distributed softbus, and designs a decentralised end-side cluster based on distributed softbus with command reply and dynamic sensing of nodes up and down. And the device information transmission mechanism based on the transmission characteristics of the distributed softbus is designed, with the help of the serialisation and deserialisation process of cJSON, to realise the collection of resource characteristics of the grouping device.

(2) Aiming at the structural characteristics of hmdfs, this paper proposes a distributed reasoning parallelisation mechanism to allocate task weights according to the current computing resource characteristics of each node. With the globally consistent access view of hmdfs, non-user nodes self-fetch text file reasoning to ensure the asynchronous nature of node reasoning.

Finally, this paper carries out transmission rate experiments on the *SendBytes* function of the distributed soft bus. After six independent repetitive experiments, this paper uses the least squares method to fit the experimental results, which ultimately results in a transmission rate of 8.9195MBytes/s for the distributed soft bus, which is a relatively faster rate compared to the 7.9MBytes/s of the TCP/IP protocol, channel utilization is improved. After that, the data parallelisation speed test was carried out on hmdfs, and it

was finally learned that there was only 4.09% speed loss in the remote reasoning, so the method proposed in this paper can accomplish the data parallelisation speed enhancement.

Keywords: **HarmonyOS distributed softbus OpenEuler embedded OS hmdfs
distributed inference**

目录

摘 要	i
ABSTRACT	iii
目录	v
第一章 绪论	1
1.1 研究背景和意义	1
1.1.1 研究背景	1
1.1.2 研究意义	1
1.2 研究内容	2
1.3 国内外研究现状	3
1.3.1 鸿蒙软总线	3
1.3.2 分布式计算框架 Spark	3
1.3.3 分布式计算任务调度	4
1.4 论文组织架构	5
第二章 分布式软总线技术研究	7
2.1 分布式软总线生态背景调研	7
2.2 分布式软总线研究	8
2.2.1 分布式软总线概述	8
2.2.2 分布式软总线功能研究	8
2.2.3 分布式软总线运行流程研究	9
2.3 基于软总线的分布式文件系统 (hmdfs) 研究	12
2.3.1 hmdfs 概述	12
2.3.2 hmdfs 部署方式	13
2.3.3 hmdfs 结构特点	13
2.4 本章小结	14
第三章 系统概要设计	15
3.1 系统框架	15
3.1.1 服务发布	15
3.1.2 节点发现	15
3.1.3 发起会话	16
3.2 指令发送和答复机制	17

3.3	动态感知	18
3.4	基于软总线的资源收集研究	19
3.4.1	收集资源目标确定	19
3.4.2	资源收集与整合	19
3.4.3	序列化与反序列化	20
3.5	基于 hmdfs 的分布式推理流程研究	21
3.5.1	任务权重分配	21
3.5.2	分布式推理算法流程	22
3.5.3	CPythonAPI 实现具体推理	23
3.6	本章小结	23
第四章	系统性能测试实验	25
4.1	性能测试实验设计	25
4.1.1	基本拓扑结构	25
4.1.2	实验方法	25
4.1.3	实验步骤	26
4.2	分布式软总线实际传输速度	26
4.3	分布式推理数据并行化速度	28
4.3.1	hmdfs 远端文件访问效率	28
4.3.2	分布式推理模型数据并行化效率分析	29
4.4	本章小结	29
第五章	总结与展望	31
	致谢	33
	参考文献	35

第一章 绪论

1.1 研究背景和意义

1.1.1 研究背景

分布式计算框架在现代计算领域中扮演着至关重要的角色，通过并行处理和资源共享，显著提升了计算效率和资源利用率。传统上，绝大多数分布式计算框架依赖于 TCP/IP 协议进行通信，采用远程过程调用 (Remote Procedure Call, RPC) 方式实现节点间的通信。例如，Hadoop 和 Spark 是两个广泛使用的分布式计算框架，均在底层使用 TCP/IP 协议进行节点间的通信，并且存在各自不同的通信框架。这些框架在大数据处理和分析方面已取得显著成果，但在面临复杂的多种设备协同工作、多种通信机制的新型应用场景时，仍然存在一些局限性。例如，它们在处理异构设备间的通信时，可能会面临协议转换效率低下和资源调度不灵活的问题。

鸿蒙是我国近年来自主研发的一款分布式操作系统，旨在打破设备之间的壁垒，实现多终端设备的协同工作和资源共享。鸿蒙系统在设计上特别强调分布式软总线的作用。分布式软总线位于系统服务层，是手机、平板、智能穿戴设备、智慧屏和车机等分布式设备的通信基座。它为设备之间的互联互通提供了统一的分布式通信能力，支持设备间的无感发现和零等待传输，极大地提升了设备间通信的效率和便利性，是我国操作系统国产化的重要实践。然而，由于鸿蒙的开发生态尚不完善，目前缺乏第三方软件的广泛支持，尤其在分布式计算领域尚未建立成熟的框架。

因此，基于鸿蒙软总线的分布式计算框架是未来值得探索的重要方向。它可以充分发挥鸿蒙分布式软总线在设备互联互通、通信协议无关等方面的优势，为分布式计算提供更加统一和高效的底层通信支撑，并且能够更好地适应新型分布式应用场景的需求，为我国分布式操作系统的发展做出积极贡献。

1.1.2 研究意义

本研究的目标是在鸿蒙系统中采用分布式软总线进行分布式计算系统的设计与实现。通过这一研究，我们不仅能够加强对分布式软总线的相关研究，填补现有研究的不足，还能为鸿蒙分布式计算领域提供重要的理论支持和实践参考。

随着新型人工智能模型对算力需求的不断增加，以及用户对高效、高质量 AI

服务的追求，传统的单终端计算方式已经无法满足需求。当前的计算模式在面对复杂的 AI 模型时，往往会因为算力不足和资源分配不均而影响性能。本研究通过探索近场设备间的自组网，使得近场设备能够参与终端的计算任务，从而实现资源的优化配置和计算能力的提升。这种方式不仅能够提升本地部署大型人工智能计算模型的能力，还能够显著提高用户的使用体验。

此外，本研究还提出了基于分布式软总线通信结构的任务计算方式，实现分布式推理的并行化。该计算方式能够在多设备之间高效分配计算任务，减轻单节点的存储和计算压力，提高整个系统的计算速度和效率。这一研究成果将为鸿蒙开发者提供有益的参考，推动鸿蒙生态系统的发展，具有重要的实践意义和应用价值。

通过在鸿蒙系统中实现分布式计算系统，我们不仅验证了分布式软总线在实际应用中的潜力，还为未来的研究和开发提供了宝贵的经验。这将有助于推动鸿蒙操作系统在分布式计算领域的发展，并为实现更高效的设备互联和资源共享奠定基础。

1.2 研究内容

本研究针对当前分布式计算框架在异构设备通信和资源调度方面的局限性，提出了一种基于鸿蒙分布式软总线的端侧集群动态管理系统。具体内容如下：

- (1) 在 OpenEuler 操作系统上，利用鸿蒙分布式软总线子系统作为通信基座。分布式软总线能够高效地连接不同通信协议和资源的设备，形成一个无主次的去中心化结构，使用户从任何节点都可以访问网络中其他节点的计算资源并获取推理结果。相对于传统的通信协议，本研究运用的分布式软总线在信道利用效率上有较大提升。
- (2) 在分布式推理过程中，借用基于分布式软总线的现有分布式文件系统，确保数据的高效传输和存储。通过鸿蒙分布式文件系统 (hmdfs)，能够实现多节点间的数据共享和任务协作。本研究所采用的传输和存储方式，时间损失小，并且实际存储分散，降低了存储压力，提高了共享效率。
- (3) 基于 Google 提供的 Bert 模型，设计和实现 FillMask 算法的分布式推理系统。充分利用软总线的高效通信和异构网络自发现自组网的优势，完成数据并行化分布式推理，最大化计算效率和资源利用率。本研究所提出的分配算法，在不借助任务队列服务的条件下完成了一定程度上的异步性，并且在传输上所损失的时间较小，实现了数据分布式对计算压力和计算时间的分散。通过预先在网络中装载 Bert 模型，完成数据并行化处理，实现多条文本的高效

推理。展示分布式软总线在实际 AI 任务分布式计算应用中的潜力和优势。

1.3 国内外研究现状

1.3.1 鸿蒙软总线

鸿蒙操作系统 (HarmonyOS) 是由华为公司开发的一款面向全场景的分布式操作系统,旨在通过分布式技术实现多设备之间的无缝协同与互联互通。自 2019 年发布以来,鸿蒙操作系统在技术进步、产业链扩展、开源生态建设以及物联网应用方面均取得了显著的进展,并得到了我国政府政策的支持。

鸿蒙与其他系统相比在于它的分布式架构,可以在多终端上进行无缝、互联互通,实现多设备协同工作^[1],并且支持与现有设备的无缝对接^[2]。迄今已经已有超过 1.5 亿搭载鸿蒙操作系统的终端设备,设备类型有手机、平板、智慧屏、手表等等^[3]。鸿蒙操作系统将电脑、手机、智慧屏、手表、车载等不同设备看成一个整体,将它们统一成一个“智能终端”操作系统。一般来说,智能家居和控制设备都是不同的系统,数据间的传输很慢,且延迟高,相互间配合起来非常不舒服,而鸿蒙操作系统是完全去中心化的,它能够植入到任何一个设备当中,把原本没有关系的设备们连接起来^[4]。

鸿蒙分布式软总线是鸿蒙去中心化分布式架构的基础。作为鸿蒙操作系统系统服务层的模块,国内研究有极大欠缺。目前,几乎没有除华为官方外的国内外研究团队针对鸿蒙分布式软总线进行实际研究,没有直接针对鸿蒙软总线的论文。官方技术文档指出,华为商用的 HarmonyOS 中分布式数据管理与分布式任务调度均基于分布式软总线实现。(注:这里的分布式任务调度指的是多设备协作、上下文感知和服务管理,而本研究所针对的任务调度指的是分布式计算资源管理。)

目前,存在有基于分布式软总线的分布式文件系统 hmdfs(Harmony Distributed File System),具备跨设备的文件访问能力,并且可以自动同步组网中多个嵌入式设备中的文件数据,支持开发者通过基础文件系统接口进行读写访问,具有高性能、低延时等优点。

1.3.2 分布式计算框架 Spark

2009 年,由加州大学伯克利分校的 AMP 实验室开发了一个名为 Spark 的大型数据处理分析项目。该项目于 2010 年正式开源。作为当时唯一支持交互式操作的集群计算引擎,Spark 迅速引起了业内的广泛关注。到 2013 年,Spark 被纳入 Apache 基金会项目^[5],并成为一个备受瞩目的开源项目。其开源社区在短短几年内的代码贡献人数就超过了 MapReduce。2014 年,Spark 被提升为 Apache 顶级项

目。自 2015 年以来, Spark 框架的流行度不断上升, 国内外许多公司开始广泛部署和使用 Spark^[6]。

Spark 计算框架的发展经历了多次迭代。2012 年, Zaharia 等人首次提出了弹性分布式数据集 (RDD) 的概念, 并在 Spark 框架中实现了这一概念, 指出在大型集群的内存中使用 RDD 可以使程序容错计算^[7]。同年, Engle 等人提出了一种粗粒度分布式内存数据分析模型, 命名为 Shark, 后来在此基础上开发了 Spark SQL。2012 年, Zaharia 等人还提出了一种高效且容错的实时流处理模型, 称为离散流 (D-Streams), Spark Streaming 在此基础上开发而成^[8]。随着 Spark 1.2 版本的发布, 引入了 ML Pipeline, 它可以将机器学习应用的多个处理过程组织起来, 构建复杂的机器学习工作流, 对分布式机器学习过程进行模块化抽象, 从而使得多个算法合并成一个 Pipeline 或工作流更加容易。

自 2013 年以来, 越来越多的开发者加入了 Spark 开源项目, 国内越来越多的公司也参与了贡献, 包括阿里巴巴、腾讯和百度等。如今, Spark 分布式计算框架已广泛应用于社会的各个领域。例如, 淘宝技术团队利用 Spark 解决计算复杂度高且需要多次迭代的机器学习算法问题, 通过 Spark 在淘宝平台上训练推荐算法, 实现商品快速推荐给特定用户。腾讯公司在广点通业务中使用 Spark Streaming 技术处理每日百亿级用户请求量, 并基于 Spark Shark 构建了日志数据快速查询系统。优酷视频基于 Spark 平台实现了视频推荐和广告推荐等功能。

此外, 腾讯公司和北京大学联合开发的 Angel 平台, 是一个大规模分布式机器学习和图计算平台, 可以在 YARN 上直接调度运行, 并支持 Spark on Angel, 集成了图计算和深度学习算法。微软发布的分布式机器学习开源函数库 SynapseML, 标准化了各种机器学习框架, 支持以不同的机器学习生态系统组件, 组成全新类别的机器学习系统。这些例子表明, 分布式机器学习是未来的研究热点。

1.3.3 分布式计算任务调度

研究者们主要集中于开发更高效的调度算法, 包括基于机器学习的方法以及启发式算法。

许多研究者致力于开发机器学习算法, D. Radhika 与 M. Duraipandian 聚焦于云计算环境中的任务调度和负载均衡问题, 结合机器学习模型特别是 SVM 算法, 可以有效地分类传入的请求, 从而减少处理时间^[9]。Wei Qu 提出了智能计算任务分配 (ICTA)。通过持续学习机制, 优化了边缘计算环境中的任务分配策略^[10]。H.Geng 团队设计了一种改进的基于强化学习的算法 DRM-DQL, 在不同的网络带宽、计算能力和 DNN 模型结构中表现出稳健性^[11]。这些算法基于机器学习或深

度学习方法，优化调度策略对不同结构的适应性，同时追求优化计算任务的执行效率。

同时，启发式算法也是在任务调度策略中的热门课题。H. Gholami 的团队在研究并行计算时，通过提出一种新的启发式算法 (LASA)，旨在优化异构分布式计算系统中的任务调度，以减少计算任务的总执行时间^[12]。类似的，WU Renbiao 团队提出一种基于截止时间的自适应调度算法。另外，也有团队针对长任务进行优化^[13]。P. Li 团队设计了一种多源 MDI (MS-MDI)，优化了跨多个源设备和工作节点的任务调度决策。所提算法能严格地控制短任务的响应速度，并使长作业的任务完成时间缩短 35%^[14]。启发式算法对于任务的执行效率方面有较强的优化能力，同样是优化任务调度策略的重要方式。

也有研究团队致力于针对 GPU 计算任务，设计特定的任务调度策略。Fan PING 团队给出了基于 GPU 计算资源的扩展贪心调度 (EGS) 算法以最大限度地利用 GPU 计算资源，并解决了 GPU 资源的闲置问题^[15]。Z.Chen 团队提出了 DeepBoot 算法，一种利用推理集群中空闲 GPU 进行训练 DLT 的方法，在提高 GPU 集群利用率和深度学习任务时十分有效^[16]。

在商用领域，云计算平台和容器编排系统（如 Kubernetes）使用分布式任务调度来管理和调度云端资源的利用率和工作负载。它们根据资源需求和约束条件，自动将任务分配给可用的计算节点，以实现高效的资源利用和任务执行。实时流处理系统（如 Apache Kafka、Apache Flink 等）使用分布式任务调度来管理和调度实时数据流处理任务。任务调度器负责将数据流分发给可用的计算节点，并确保任务按时处理和响应，以实现实时数据分析和处理

分布式计算任务调度领域的发展趋势指向了几个关键方向。首先，人工智能和机器学习技术的应用被视为优化任务调度策略的重要途径。启发式算法也是在任务调度策略中的热门课题，在优化任务的执行效率方面有较强能力。同时，也有团队致力于针对 GPU 计算任务，设计特定的任务调度策略。这些技术有助于提高调度决策的智能化和自动化水平。

1.4 论文组织架构

第 1 章 绪论。整体上介绍了“基于鸿蒙软总线的端侧集群动态管理设计实现”课题的意义、目的，研究了国内外关于分布式软总线和分布式计算任务调度的相关研究及发展趋势，并对本次研究工作进行详细安排。

第 2 章 对分布式软总线的研究。介绍了分布式软总线的基本能力，以及分布式软总线在运用时的四个步骤的基本原理，同时介绍了基于分布式软总线的分

布式文件系统的具体功能和部署方式。

第 3 章 系统原型概要设计。介绍了本研究中的原型系统的基本框架设计，介绍了关键的答复机制的设计模式，以及各个重要的子模块和相关的重要技术细节。

第 4 章 对系统各部件性能的测试。主要对分布式软总线 *SendBytes* 函数的传输速度进行了测试，并且采用最小二乘法拟合计算出了其实际的信道速度。之后对 *hmdfs* 在分布式推理时的效率进行了测试，并且对实际的推理时间的优化进行了推导。

第 5 章 总结与展望。

第二章 分布式软总线技术研究

2.1 分布式软总线生态背景调研

目前,参与分布式软总线生态的有 OpenHarmony 和 OpenEuler 两种操作系统: OpenHarmony 主要面向强交互等需求的智能终端、物联网终端和工业终端,是当前商用的 HarmonyOS 的开源版本。OpenEuler 主要面向有高可靠、高性能等需求的服务器、云和嵌入式设备,有望成为国产服务器端 Linux 的主流^[17,18],二者各有侧重。并且 OpenEuler 平台易于使用、成本低廉,^[19]同时具备强大的硬件平台适应性,支持多处理器架构、统一和开放的操作系统,已经存在面向微型 ARM 主板的嵌入式版本,推动软硬件应用生态繁荣发展^[20]。

同时, OpenHarmony 和 OpenEuler 通过软总线技术,可以实现两种操作系统之间的生态互联,更加扩展了分布式软总线的使用范围和应用场景,跨平台开发并一键部署在这里成为了可能。



图 2.1 OpenHarmony 示意图

软总线位于 OpenHarmony 系统服务层（系统服务层是 OpenHarmony 的核心能力集合,通过框架层对应用程序提供服务),如图2.1所示。分布式软总线属于系统基本能力子系统集,为分布式应用在多设备上的运行、调度、迁移等操作提供了基础能力。OpenHarmony 的分布式数据管理和分布式任务调度均基于分布式软总线实现。是华为鸿蒙生态的基础性技术。

在对硬件平台适配的研究中,本文曾调研过 RK3588、Atlas、RK3568、树莓派等嵌入式平台。其中树莓派应用广泛、成本较低,而 OpenEuler 硬件适配完善,在服务器、嵌入式领域同样应用广泛。最终本文将树莓派与 OpenEuler embedded (OpenEuler 面向嵌入式平台的版本) 相结合,完成本毕业设计的硬件呈现。

2.2 分布式软总线研究

2.2.1 分布式软总线概述

分布式软总线为应用和系统提供近场设备分布式通信的能力，提供不区分通信方式的设备发现、连接、组网和传输功能。这里的不区分通信方式，指的是软总线具备将 WIFI，Ethernet，BLE 等通讯方式进行统一管理，抽象成统一的通信 API 供开发者使用，开发者不必关注通信方式的差异和细节。

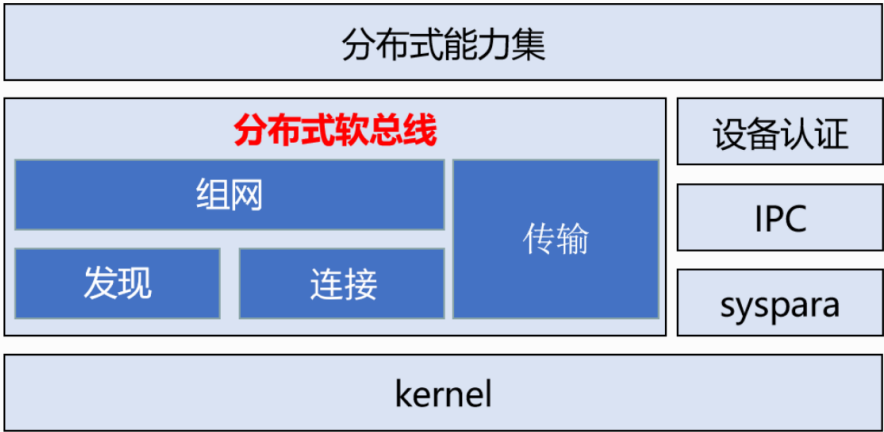


图 2.2 分布式软总线架构示意图

2.2.2 分布式软总线功能研究

分布式软总线需要在 WIFI，Ethernet，BLE 等通讯方式已经建立连接的条件下进行组网，在软总线运行时，只借用传统通信方式的信道。

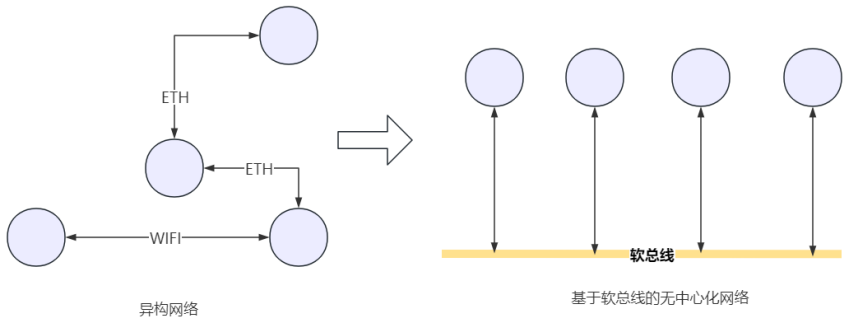


图 2.3 软总线抽象示意图

如图2.3所示，近场设备的连接并不会一定符合某种特定的拓扑结构，比如说都在同一个路由器的局域网下，或是都采用 Ethernet 链路连接。而软总线的最大优势，就是将左图的任意异构网络均纳入统一管理，抽象成类似总线的通讯方式。所有在软总线上挂载的设备，均会获得自己独特的 *NodeBasicInfo* 结构体，作为区分各个设备的标签。

```
typedef struct {
    char networkId[NETWORK_ID_BUF_LEN];
    //NETWORK_ID_BUF_LEN = 65
    char deviceName[DEVICE_NAME_BUF_LEN];
    uint16_t deviceId;
} NodeBasicInfo;
```

其中,最重要的属性是 `networkId`, `networkId` 是一个实际长度为 64 位,数字与字母随机组合的字符串。在每次分布式软总线服务被拉起时自动随机分配。需要强调的是,每次软总线服务被拉起时,分配的 `networkId` 都不相同。并且,可以完整获取 *NodeBasicInfo* 的接口仅存在于用户端的发现模块。这些设计方式保证了软总线服务在运用时具备较高的安全性。

分布式软总线通信发起方式为,在运用发现模块获取当前软总线下的设备信息后,就可以开始进行通信。目前,软总线提供了四种通信方式:字节通信、信息通信、流式通信、文件通信,通过以 **Send** 命名前缀的函数族实现。前三种通信方式在完成组网后的通信时序,需要在与对端发起会话后进行。并且,需要在完成通信之后回收会话资源。而对于文件传输来说,需要先设置文件传输监听服务,并且在完成传输后回收监听服务,其余过程与普通通信方式相同。

在传统类 **linux** 系统网络编程中,信息接收是通过 **listen** 系统调用监听某个特定端口来实现的。同时,发送端也要经过这个端口发送,才能完成一次通信。这种绑定端口的通信方式缺乏灵活性,并且信息交互相对复杂。在分布式软总线当中,信息的接收通过回调函数实现。在 C 语言中,回调函数是一种函数指针的应用,其为 C 语言提供了一种灵活的方式来执行函数。当软总线中的点对点通信发生时,发送端调用 **Send** 函数族向对端发送信息。而接收端接收到信息后,立即触发以 **Received** 命名后缀的函数族的回调。在这个回调函数中,软总线将提供发送端的信息,以及接收的内容。用户可以自行定义对消息的操作,或者是同时发送答复,无需再次启动通信的准备工作。这种借助回调函数的接收方式,强化了软总线应用的灵活性和效率。同时,回调函数也是软总线具体实现中大量应用的关键技术。

2.2.3 分布式软总线运行流程研究

首先需要明确的是,软总线在每个设备上部署时均分两个模块,**Server** 模块是支撑软总线的核心模块,而开发者开发的均为 **Client** 模块。其拓扑结构如图2.4所示。

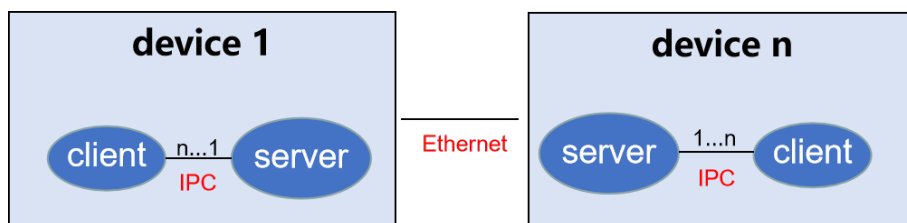


图 2.4 分布式软总线部署拓扑结构示意图

第一阶段是发布阶段，发布端的主要运用的 CoAP 协议，CoAP 是受限制的应用协议（Constrained Application Protocol, CoAP）。在当前由 PC 机组成的世界，信息交换是通过 TCP 层和应用层协议 HTTP 实现的。对于物联网小型设备而言，实现 TCP 和 HTTP 协议显然是一个过分的要求，而 CoAP 这种轻量级协议可以更好的适配。从技术实现上，CoAP 协议底层基于 UDP 的广播协议，主动发现方通过在网络中广播特定的报文，等待其他软总线设备予以响应，从而实现设备之间的相互发现。

在具体的实践中，只要软总线服务被拉起，各个节点中的 Server 模块便会自动广播后完成组网，Client 模块无需额外操作便可以与之交互。

该模块时序图如图 2.5 所示。外围设备首先向软总线客户端发出请求，由于软总线客户端此时无法直接调用服务端的服务，需要首先向服务端代理 (server_proxy) 发出请求。请求完成后，服务端代理 (server_proxy) 再向服务桩 (server_stub) 发出请求，最后调用到软总线服务端。这个过程被称为代理和桩 (proxy-stub) 设计模式，被广泛应用于鸿蒙操作系统进程通信中。

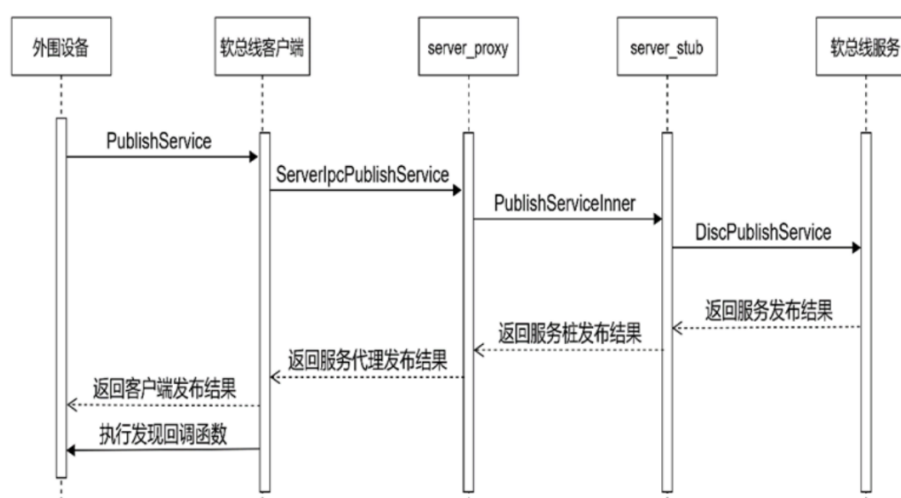


图 2.5 软总线发布示意图

第二阶段是发现阶段，与发布模块相似，发现模块时序图如图2.6所示。连接请求依次调用软总线客户端、服务端代理 (server_proxy)、服务桩 (server_stub) 最后

调用到软总线服务。调用完成后，依次返回给外围设备，最后根据调用的执行结果执行相应的回调函数，这里主要的回调函数有发现设备回调 *OnDeviceFound*、发现失败回调 *OnDiscoverFailed*、发现成功回调 *OnDiscoverySuccess* 回调。

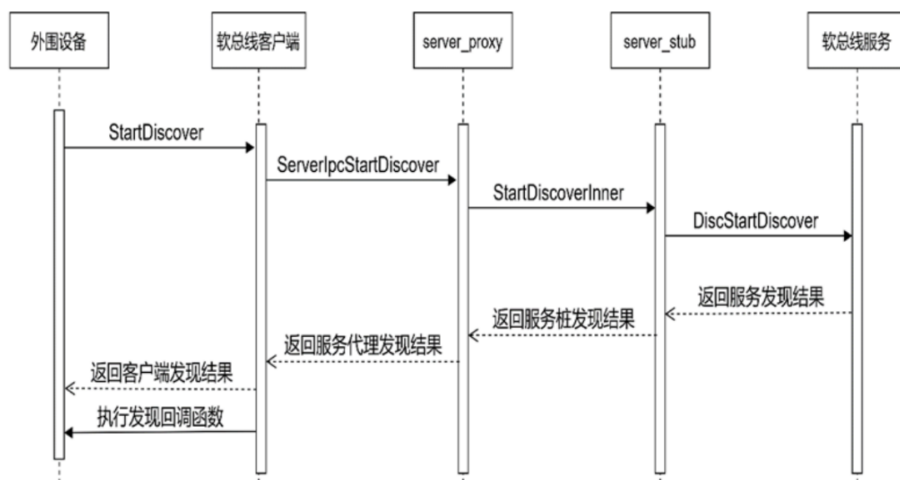


图 2.6 软总线发现示意图

第三阶段是组网阶段，设备组网主要的功能是提供统一的设备组网和拓扑管理能力，为数据传输提供已组网的设备信息。和发现与连接模块不同，组网模块的处理流程较为复杂，其时序图如图2.7所示。

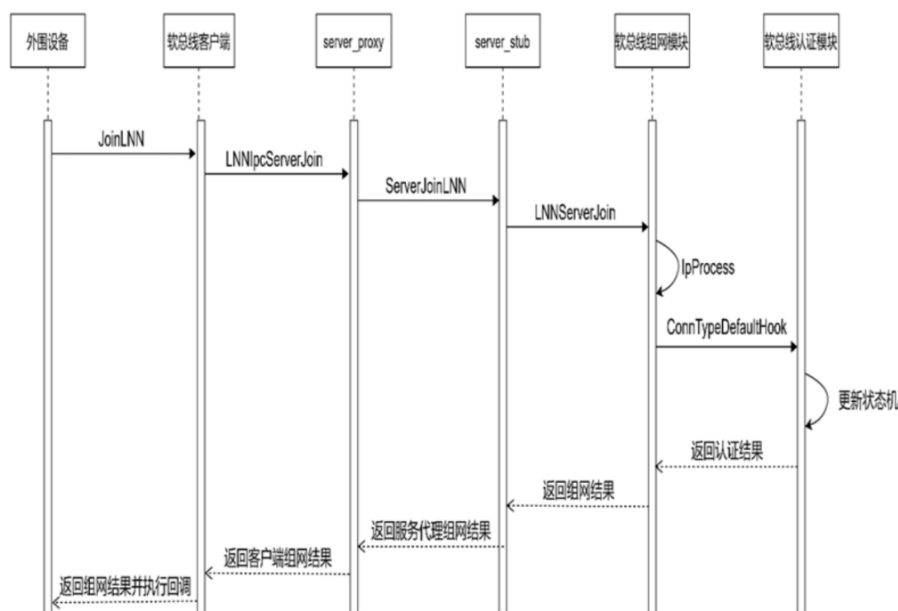


图 2.7 软总线组网示意图

认证完成后，会将认证信息保存，并依次返回执行结果，然后通知客户端启动回调函数。这里的回调函数主要包括设备上线 (*onNodeOnline*)、设备下线

(*onNodeOffline*)、设备基本信息变化 (*onNodeBasicInfoChanged*) 三种回调函数，让分布式软总线模块能够更加方便的监视网络情况。

第四阶段是传输阶段，假设外围设备 A 和 B 想要传输数据，设备 A、B 必须创建会话服务，并执行注册回调。这里的回调函数主要有会话开启、关闭和有数据到达被调用。当外围设备打开会话后，外设会调用相应的回调函数做好传输数据的准备，之后就可以传输数据。

与传统 P2P 传输有着较大不同，多设备场景下资源可调度、可利用空间更加广阔，传输数据的类型也有字节、文件、消息和流数据，这几种传输方式在步骤上有着一定的差别，但大体原理相似。传输完成后，外围设备关闭会话，执行相应的回调函数，整个传输过程结束。

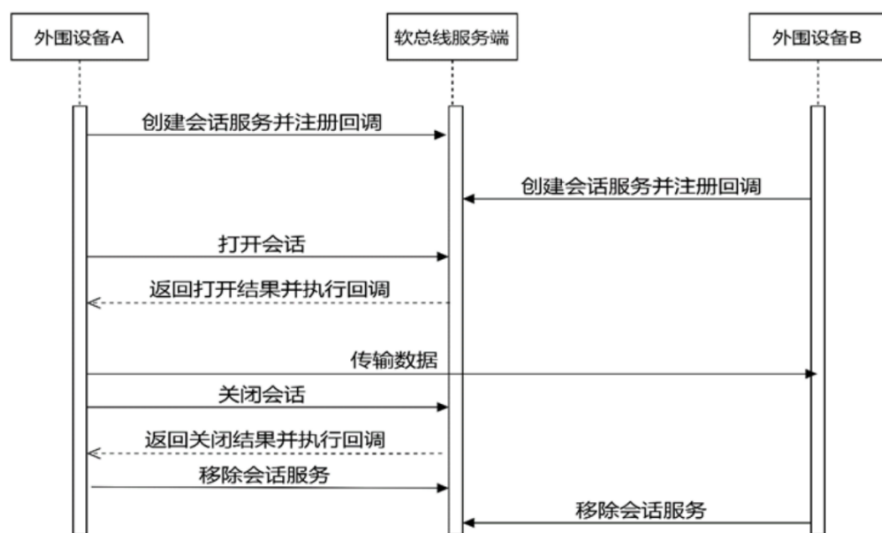


图 2.8 软总线组网示意图

2.3 基于软总线的分布式文件系统 (hmdfs) 研究

2.3.1 hmdfs 概述

hmdfs 在分布式软总线动态组网的基础上，为网络上各个设备结点提供一个全局一致的访问视图，支持开发者通过基础文件系统接口进行读写访问，具有高性能、低延时等优点。

其包括如下 2 个核心模块：

- **distributed_file_daemon**：分布式文件管理常驻用户态服务，负责接入设备组网、数据传输能力，并负责挂载 hmdfs。
- **hmdfs(Harmony Distributed File System)**：分布式文件系统核心模块，是一种面向移动分布式场景的、高性能的、基于内核实现的、堆叠式文件系统。

2.3.2 hmdfs 部署方式

在 OpenEuler 23.03 LTS SP3 版本中，hmdfs 已经可以作为 dnf 指令的软件包安装，但是安装后也有具体的部署步骤要求。

完成 hmdfs 的完整部署并生效，需要进行如下的步骤：

1. dnf 指令拉取并安装 hmdfs 和 filemanagement_dfs_service 两个软件包；
2. 插入 hmdfs.ko 内核模块，使得 OpenEuler 可以识别文件系统；
3. 在/etc/SN 文件中写入当前系统唯一的 SN 号（可以是任意字符串）；
4. 挂载 hmdfs 目录（为了与 OpenHarmony 对接，源目录与挂在上的目录名称固定为/data/service/el2/100/non_account 到 /mnt/hmdfs/100/non_account）；
5. 启动 start_service.sh 脚本下的 dfs_service 服务。

2.3.3 hmdfs 结构特点

挂载成功后，会产生如下的目录结构。

```

├─ device_view
│   ├─ {networkId1}
│   ├─ {networkId2}
│   └─ local
└─ merge_view

```

目录下会有两个文件视图：device_view 是分设备的视图，其中 local 是本地文件视图，networkId1 是软总线 networkId 为 networkId1 的设备的文件视图，networkId2 是软总线 networkId 为 networkId2 的设备的文件视图；merge_view 是合并视图，多个设备的文件都在这同一个目录。后续需要跨设备进行文件操作，只需要操作 device_view 下面远端设备目录下的文件即可，也可以针对 merge_view 进行统一管理。

如果出现多个设备文件名相同的情况，在 merge_view 中将会被重命名，但是每个设备中的文件不会被重命名。如果多个设备文件名冲突，以接入本设备 ID 为顺序，显示设备 ID 小的同名文件，其他文件被依次重命名。冲突文件显示 _conflict_dev 后依次加 id，id 从 1 自动递增。同名目录之间仅融合不存在冲突，文件和远端目录同名冲突，远端目录后缀加 _remote_directory。

需要指出的是，由于 hmdfs 底层采取 UDP 协议，UDP 本就是轻量级协议，因此在传输较大文件时会有不稳定的现象发生。

2.4 本章小结

介绍了分布式软总线的基本能力和功能研究，以及分布式软总线在运用时的四个步骤的基本原理，同时介绍了基于分布式软总线的分布式文件系统的具体功能和部署方式。

第三章 系统概要设计

本系统采取各个模块解耦合编程，便于后续调试改进。系统整体采取去中心化设计，用户可以从任意组网节点中观察并发送指令，个节点之间地位等同。重点解决三个问题：

- (1) 设计基于软总线的指令发送和答复机制。
- (2) 设计基于软总线的序列化与反序列化机制，完成资源的收集；
- (3) 设计基于 **hmdfs** 的分布式推理流程，实现高效的数据传递，保证各个节点之间的异步性。

3.1 系统框架

在基本框架中，本系统实现了节点发现，服务发布，会话发起。这几个模块共同组成了通信框架，并且形成了后续重要的答复机制，为后续的指令发送奠定基础。整体框架如图3.1所示。（拉起 **hmdfs** 服务的方式已经在2.3.2介绍。并且拉起 **hmdfs** 服务的同时，软总线服务也会同时拉起，因此 **hmdfs** 的拉起没有在图中呈现）

需要强调的是，本系统为去中心化的结构，各自在软总线上的地位等同，因此，所有节点应该在发布服务后再执行发现。

3.1.1 服务发布

本系统中，服务发布的总流程被封装成 *PublishServiceInterface*，在完成 *PublishInfo* 结构体初始化后，将系统自定的对应回调函数注册到 *IPublishCallBack* 结构体中的发布成功和发布失败函数指针，这种成功或失败的回调函数主要用于系统日志，实际不发生其他系统动作，后续类似的回调函数也是如此。最后，调用 *PublishService API* 完成服务发布。同时，也完成对应的停止发布函数，完成该资源的回收。

3.1.2 节点发现

本系统中，节点发现的总流程被封装成 *DiscoveryInterface*，与服务发布类似，需要对 *SubscribeInfo* 结构体进行初始化，然后将系统自定的对应回调函数注册到 *IDiscoveryCallBack* 结构体中的设备发现、发现成功和发现失败函数指针。也完成对应的停止发现函数，完成该资源的回收。

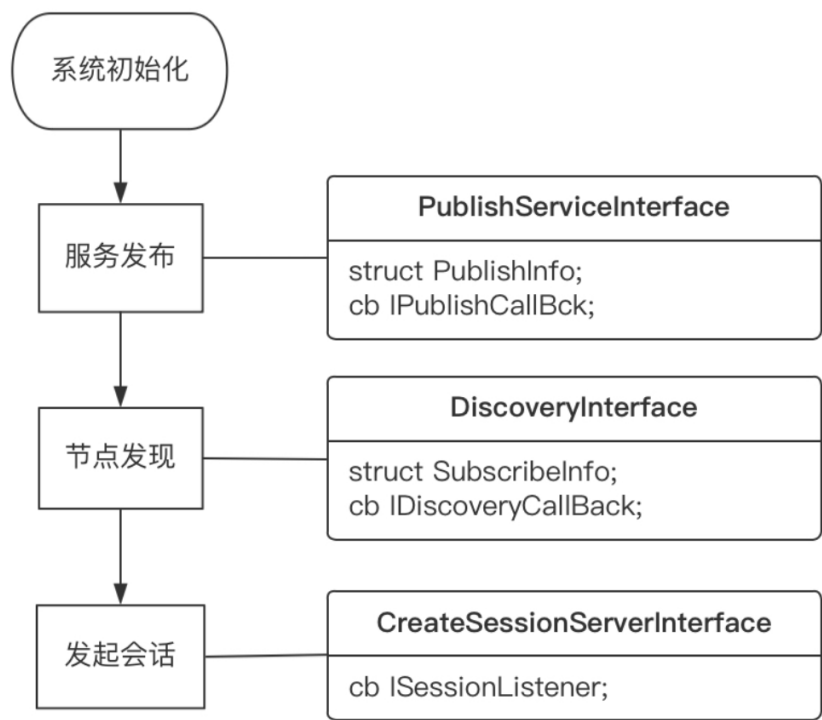


图 3.1 系统框架示意图

其中，设备发现回调是当前接口中最重要的回调函数。在本模块最终调用 *StartDiscovery* API 后，本端触发回调，假如有多个节点被发现，则多次触发该回调。该回调将返回某个特定节点的 *DeviceInfo* 结构体，其中包含的重要信息，包括设备的 *networkId*，参与软总线组网的通讯方式和区分标志。（比如某个节点采用 WIFI 形式参与组网，*DeviceInfo* 结构体将包含该节点的组网方式为 WIFI，同时给出 IP 与端口号。）

然而，由于多次触发回调的特性，且触发顺序和时机对用户不公开，设备发现回调的功能依然是系统日志，不发生实际上的系统动作。

另外需要强调的是，这里的发现应该被理解为一个持续过程，只有保持节点发现接口打开，才能进行后续的设备信息汇总模块。

3.1.3 发起会话

本系统中，发起会话之前需要创建会话服务器，被封装成 *CreateSessionServerInterface*，将系统自定的对应回调函数注册到 *ISessionListener* 结构体中的发布成功和发布失败函数指针。另外，上文中提到的 *Received* 函数族也在此处注册。最后，调用 *CreateSessionServer* API 完成服务器创建。也

完成对应的服务器关闭函数，完成该资源的回收。

3.2 指令发送和答复机制

在保持发现的条件下，调用 *GetAllNodeDeviceInfo* API 之后，便可以获取到当前组网的全体设备信息。*GetAllNodeDeviceInfo* 需要传入一个 *NodeBasicInfo* 结构体的二维指针 *dev_map*，第一维保存某个特定设备的信息，第二维将第一维全体保存便于索引。同时需要传入一个预先定义的整数参数指针，设备总数就保存在这个指针当中。指针结构如图3.2所示。

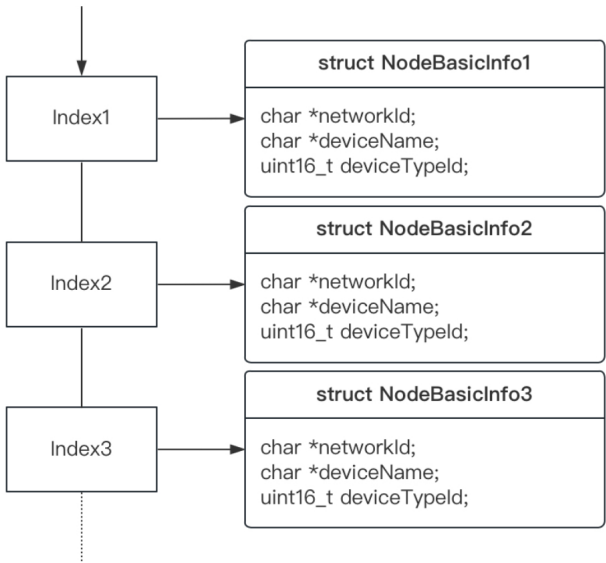


图 3.2 设备图二维指针示意图

基于该设备图，本研究完成节点指令发送和答复机制的设计。首先，在每次通信开始前，需要发起会话，本系统将其封装成 *OpenSessionInterface*，需初始化 *SessionAttribute* 结构体，最后调用 *OpenSession* API。需要注意的是，*OpenSession* API 里的 *local_session_name* 和 *target_session_name* 应该相同。通信完成后，关闭会话，完成资源回收。

答复机制结合上述过程，基于字节传输 **Received** 函数族，完成答复机制。如图3.3所示。在后续的示意图中，不再展示答复流程，仅标注答复来代表该流程。

答复机制需解决两个核心问题：第一，如何确认哪个消息是指令，哪个消息是答复；第二，何时进行资源回收。

本系统充分利用 **Received** 函数族，设计指令字与答复字的共识机制。所有节点认为，只有指令字中携带“get”关键字，只有答复字中携带“{”关键字。

发送指令字的设备发起会话，并向对端发送指令字，对端接收指令字发生系

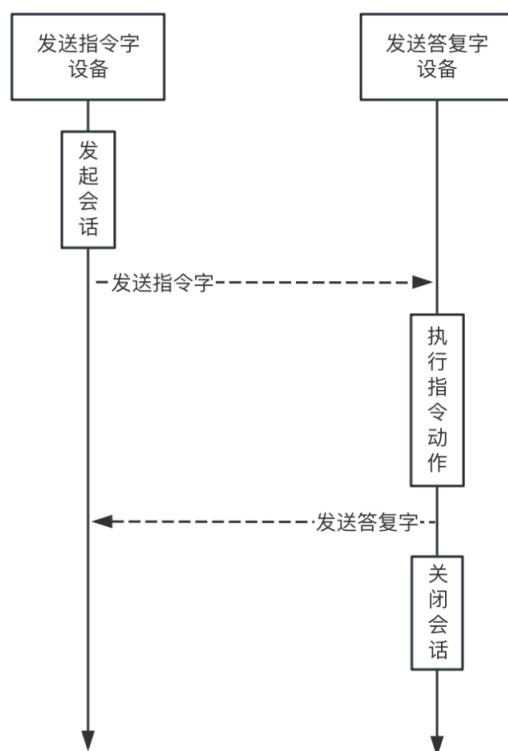


图 3.3 答复时序示意图

统动作后，向发送指令字的设备发送答复字，关闭会话。因此，发送指令字的设备发起会话，指令字发回后关闭会话。会话创建与接收实质上发生在两个不同的设备上。

3.3 动态感知

本系统对节点动态感知的设计，采用了 `select` 系统调用。采用分布式软总线进行组网的设备，互相之间地位相同，并不存在 TCP/IP 特有的 `Sever/Client` 模型，这就意味着，假定某个节点非正常下线，该节点将无法向其他用节点通知自身的离线。正由于对于系统容灾能力的考量。假定某个设备出现了灾难性的损坏，软总线的 `OffLine` 回调便不能实时感知到。

`select` 系统调用是 UNIX 和类 UNIX 操作系统中用于监视多个文件描述符以检测其是否准备好进行输入或输出操作的一种功能。这个系统调用允许程序在单个线程中同时处理多种输入/输出活动，这对于开发需要处理多个连接或输入源的应用程序（如网络服务器或同时从多个文件读取数据的应用）非常有用。

使用 `select` 调用时，开发者可以指定三个集合的文件描述符：读操作准备好的文件描述符集合、写操作准备好的文件描述符集合和异常条件出现的文件描述

符集合。此外，还可以设置一个超时值，用来限制 `select` 等待这些文件描述符变为可用状态的时间，使得程序在没有任何文件描述符准备就绪时不会无限期地阻塞。

本系统采用 `select` 异步监控用户操作并定义超时机制。设定五秒超时，每次超时，调用 `GetAllNodeDeviceInfo` API 对组网设备进行更新，感知当前组网状态。这样，即便有设备因灾难性损坏，其余节点依然可以感知到该设备的下线。

3.4 基于软总线的资源收集研究

3.4.1 收集资源目标确定

首先从需求上看，本系统应能做到观察节点的计算资源信息，并且对这些资源进行评估。

本研究考虑到一次人工智能推理的两个基本过程：模型加载和推理。模型加载时，需要有充分的内存资源可以载入模型文件。而在推理时，当前的 CPU 负载率会极大的影响到实际推理的效率（由于我们采用的设备是仅有 CPU 的树莓派，因此不会考虑 GPU）。因此，最终确定将 CPU 负载、内存相对使用率和内存绝对使用率作为评估某个设备计算能力的关键指标。

需要注意的是，从操作系统的层面，无法实时感知当前 CPU 的真正算力，因此这些指标仅用于判断当前设备是否具备参与分布式推理的能力，以及后续获得的计算任务量的权重判断。

3.4.2 资源收集与整合

OpenEuler 采用 Linux 内核，是类 Linux 操作系统。因此，系统资源保存在 `/proc` 目录下，CPU 负载率可以读取 `/proc/loadavg` 文件的内容来获取，关于内存的信息保存在 `/proc/meminfo` 中。本系统将这些信息整合进一个结构体中。

```
typedef struct {
    float  cpu_load;                // 系统负载
    float  memory_usage_percent;    // 内存利用率（百分比）
    float  memory_usage_absolute;   // 内存绝对利用量
    float  weight;                  // 权重得分值
    char  networkId[NETWORK_ID_BUF_LEN];
} SystemInfo;
```

每次系统收到资源收集指令时，都会对该结构体进行更新并暂存，本系统调用 `GetSystemInfo` 函数完成自身资源更新，以备后续的其他操作。同时，用户端

程序保存一个列表 `all_info` 用于保存全体设备的 *SystemInfo*。

3.4.3 序列化与反序列化

序列化与反序列化是分布式计算中的重要概念，主要用于数据的存储与传输。这两个过程解决了如何将数据结构或对象状态转换成可存储或传输的格式的问题，以及如何从这种格式恢复回原始的数据结构或对象状态。

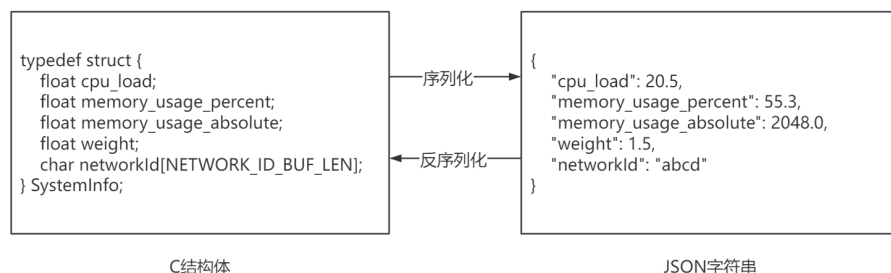


图 3.4 序列化与反序列化示意图

序列化是将数据结构或对象状态转换为一个字节序列的过程，可以存储到文件中，或者通过网络传输到另一个系统。这一过程的核心在于将内存中的复杂数据结构转换为一个线性的格式，使得数据可以在不同的硬件和操作系统之间进行交换。在本系统中，*SystemInfo* 被序列化为一个 JSON 字符串，结构体的成员属性按照同名键值对保存在 JSON 中，并通过 *SendBytes* API 进行发送。

选择 JSON 的原因，在于它的这种结构直观，易于人类理解和编写，同时也易于计算机解析和生成。此外，JSON 的数据格式虽然源于 JavaScript，但几乎所有的现代编程语言都提供了解析 JSON 的库，使得 JSON 成为跨语言、跨平台数据交换的优选方案。并且，*SendBytes* API 具备一次传输长度最多为 984 的字符串的能力。因此，软总线可以一次性传输一个较大规模 JSON 字符串，为本系统序列化传输变成可能。另外，*SystemInfo* 结构体的属性均为简单的键值对，便于序列化为一个 JSON 字符串。

在本系统中，具体采用了 cJSON 完成这一过程。cJSON 是一个轻量级的 JSON 解析器和生成器，专为 C 语言设计。它由 Dave Gamble 创建，目的是提供一个简洁高效的方法来处理 JSON 数据，特别适合嵌入式系统和资源受限的环境。cJSON 的设计理念是易于集成、使用和维护，同时保持代码的轻量 and 性能。OpenEuler 中集成有 cJSON 的软件包便于使用。

反序列化是序列化的逆过程，它将从文件中读取的字节序列或接收到的字节流转换回原始数据结构或对象。这个过程对于数据恢复至关重要，尤其是在数据需要跨网络传输或在程序重启之间持久保存时。通过反序列化，系统可以重

建对象和数据结构，以便它们可以在程序中像原始状态一样使用。本系统设计 *ParseSystemInfo* 函数用于反序列化，按照 JSON 中的标签名，对 JSON 字符串中的结构体进行还原，并保存在本地的 *all_info* 中。

本系统在上述指令字与答复字的共识部分的共识机制由此产生。所有节点的统一共识认为，指令是一个某种 *get* 操作，而答复以 JSON 的形式进行。

3.5 基于 hmdfs 的分布式推理流程研究

本系统进行的分布式推理样例，是基于 Bert 模型的填充掩码推理，完成的是数据并行化分布式推理。

BERT (Bidirectional Encoder Representations from Transformers) 是一种基于 Transformer 模型架构的自然语言处理预训练模型，由 Google 的研究团队在 2018 年提出。BERT 的核心创新是利用大量未标注文本通过双向训练的方式，来更好地理解语言的上下文。

填充掩码推理 (fill-mask inference)，是 Bert 的一种特别的使用模式，直接源于 BERT 的掩码语言模型训练任务。在这种模式下，模型接收到一个包含有一个或多个掩码标记（通常是"[MASK]"）的句子，模型的任务是预测这些掩码标记在语境中最可能代表的词语。这种能力使 BERT 在处理填空题或进行某些类型的文本生成时表现出色。

例如，对于句子 "The capital of France is [MASK].", BERT 模型将利用其对语言的理解，检查词语 "capital" 和 "France" 的关系，并预测出 [MASK] 最有可能的填充选项是 "Paris"。这种能力的实现依赖于 BERT 在预训练期间学习到的大量语言知识和句子结构理解。

3.5.1 任务权重分配

本研究在进行分布式推理之前，首先会进行任务权重分配。任务权重在获取系统信息时实时更新。任务权重的分配分两个基本阶段。

第一阶段：过滤阶段，本系统认为，如果某个设备的 CPU 负载过高，或者内存绝对量未达到模型加载需求（由于系统在初始化时就已经进行模型加载尝试，如果加载失败，权重也视为 0），则权重直接置 0。

第二阶段：分配阶段，本系统认为当前设备符合参与计算的条件，则分配一个由 CPU 负载和内存相对使用量加权产生的权重作为计算能力的体现。计算公式如下。

$$W = \alpha(1 - C) + \beta(1 - R) \quad (3-1)$$

公式中, α, β , 分别为预定义的权重值, C 为 CPU 负载, 为一个百分率值, R 为内存相对使用率, 同样为百分率。结果 W 为产生的绝对权重, 经过用户观察节点的归一化之后, 作为数据并行化的指标。可以看出, CPU 负载越低, 内存相对使用率越低, 自身所获取的权重越高, 就需要承担更加繁重的计算任务。

本系统认为, CPU 负载在这两个因素中更加影响速度, 因此设定 $\alpha = 0.8, \beta = 0.2$

3.5.2 分布式推理算法流程

本研究的分布式推理模式, 是基于 hmdfs 的结构设计的。它包含两个基本过程, 切分文本和节点自取文本推理。节点自取文本推理解除了节点之间的互相依赖关系, 实现了节点间较强的异步操作。这个过程如图3.5所示。

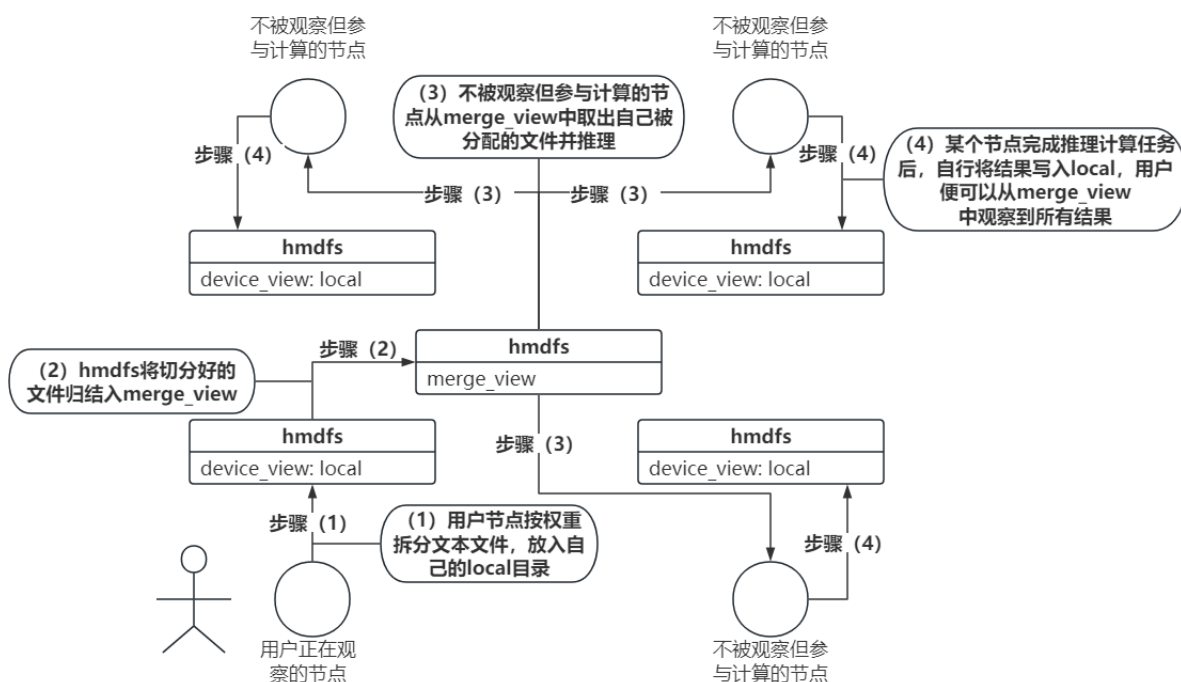


图 3.5 分布式推理过程示意图

首先, 代码计算了所有设备的权重总和 `total_weight`, 然后对每个设备的权重进行了归一化处理。归一化是通过将每个设备的权重除以总权重来实现的, 确保所有权重加起来等于 1。

之后, 按归一化权重切分文件, 源文件是一个由 100 行 (也可以其他数字, 本样例采用 100 行) 待推理文本组成的文本文件。产生的文件被命名为 `input_networkId`, 这里的 `networkId` 是不确定的, 根据当前网络情况里节点的

`networkId` 来覆盖。便于其他节点按照自身的 `networkId` 自取文件推理。

完成文件切分并放入 `hmdfs` 后，用户观察的节点发送 `get_process_python` 指令，这是一个无答复的指令字。所有节点收到指令后立即自取文件推理，并将结果放入自己的 `local` 目录当中。所有节点自取文件，互不干涉，实现了异步操作。

3.5.3 CPythonAPI 实现具体推理

CPythonAPI 是一种允许开发者使用 C 语言来扩展 Python 语言功能或优化性能的编程接口。该 API 是 CPython 解释器的一部分，即 Python 的一个主要实现，提供了一组功能丰富的工具来创建新的 Python 类型、调用 Python 对象以及执行 Python 代码等。

通过 CPythonAPI，开发者能够直接从 C 语言中操作 Python 对象，包括创建 Python 对象、调用 Python 函数和方法、以及访问 Python 数据结构等。这一功能特别适用于需要优化性能的情景，尤其是在处理计算密集型任务时，因为 C 语言提供了更接近硬件层的控制能力和高效的执行速度。

本系统采用 CPythonAPI 调用 `python` 脚本完成推理。在系统初始化时调用 `Py_Initialize` API 启动 Python 解释器，并且使用 `PyObject_CallObject` API 预载模型备用，只有在每次收到指令后才进行实际推理。在系统退出时，释放模型对象并关闭解释器。

3.6 本章小结

本章详细描述了系统的概要设计，包括初始化设计、答复机制、节点指令发送、资源收集、分布式推理，以及动态感知等核心模块。整个设计强调了模块化和解耦合，确保了系统的灵活性和可维护性。通过去中心化的架构，系统支持节点发现和服务发布等功能，为设备间的有效通信提供了基础。此外，本章还介绍了系统的答复机制和资源管理策略，以及如何通过分布式处理来优化任务执行效率和资源利用率，以及分布式推理时的任务分配策略。

通过对关键技术和操作的深入分析，本章为理解系统的工作原理和实现细节提供了清晰的视角。这些设计选择和实现策略共同支撑了系统的功能需求，确保了高效、可靠的运行性能。

第四章 系统性能测试实验

4.1 性能测试实验设计

4.1.1 基本拓扑结构

本次针对传输的测试在如图4.1的拓扑结构下进行。由一个路由器将两个树莓派组成局域网，一个树莓派采用 Ethernet 连接，一个树莓派采用 WIFI 连接。路由器的中转延迟可以用来模拟实际条件下网络不稳定的可能性。

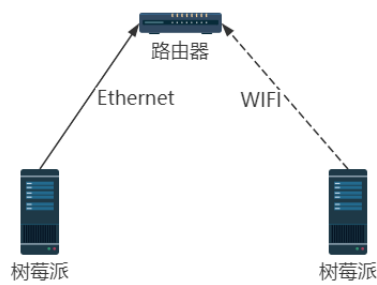


图 4.1 性能测试拓扑结构图

本次实验使用的主要设备包括两台树莓派（型号 4B+），一台配备 Ethernet 连接，另一台通过 WiFi 连接。所有设备均连接到型号为 TP-Link TL-WR886N 的路由器上。树莓派操作系统为 OpenEuler embedded 22.03 LTS SP3。

设备种类	
软件	OpenEuler embedded 22.03 LTS SP3
	communication_dsoftbus 1.0.0-1.oe2203sp3
	Harmony distributed file system 1.0.0-1.oe2203sp3
硬件	树莓派 4B+
	TP-Link TL-WR886N 路由器

表 4.1 实验所用设备列表

4.1.2 实验方法

本研究在测量时间时，采用了 `<sys/time.h>` 头文件中 `gettimeofday` 函数。该函数用于获取当前的时间，这个时间包括自 1970 年 1 月 1 日（即 Unix 纪元）以来的秒数和微秒数。`timeval` 结构体用于表示时间，`gettimeofday` 函数便借助该结构体获取时间，其定义同样在 `<sys/time.h>` 中。

```
struct timeval {
    time_t      tv_sec;          /* 秒 */
    suseconds_t tv_usec;        /* 微秒 */
};
```

在本研究中，采用毫秒（ms）作为记录时间的基准单位。而由于 `timeval` 结构体的特殊结构，产生一个毫秒时间需要经历如下步骤。

首先，计算 `end_time.tv_sec` 和 `start_time.tv_sec` 的差值。由于 `tv_sec` 代表自纪元以来的完整秒数，这个差值给出了两个时间点之间完整秒的差距。然后将这个秒差值转换为毫秒（乘以 1000）。

接下来，计算 `end_time.tv_usec` 和 `start_time.tv_usec` 的差值，这给出了微秒级的时间差。由于这个差值是微秒，所以需要将其转换为毫秒（除以 1000）。

上述两个时间相加为最终的毫秒数。这种处理方式确保了时间差计算的完整性和精度，即使跨越了秒边界（例如，从 `tv_sec` 的 1 秒 999,999 微秒到 2 秒 1 微秒）也能准确反映微秒级的时间变化。

4.1.3 实验步骤

1. 采集的时间数据首先经过初步处理，计算每次测试的毫秒数。
2. 为了减少偶然误差，每种字节大小的传输重复五次，取平均值作为最终结果。
3. 使用 **Matlab** 进行统计分析和图形化表示，以确定 *SendBytes* 函数的性能特征。
4. 拟合的直线帮助理解传输时间如何随字节大小的增加而变化，并计算实际传输速率。

4.2 分布式软总线实际传输速度

分布式软总线给出了四种传输方式：字节、信息、流、文件。本节针对 *SendBytes* 函数传输不同长度字符串的运行时间进行了多次重复实验，并最终取每种字节数传输时间的平均值。最终数据如表4.2所示。

观察传输时间的平均值可以发现，传输时间并非与字节数成正比例，而是具有线性关系。这是因为，*SendBytes* 函数在传输前依旧有准备时间，这段时间在一次测量中是无法被提取测量的。但是在每次运行时，*SendBytes* 的准备时间可以视为大致相同。

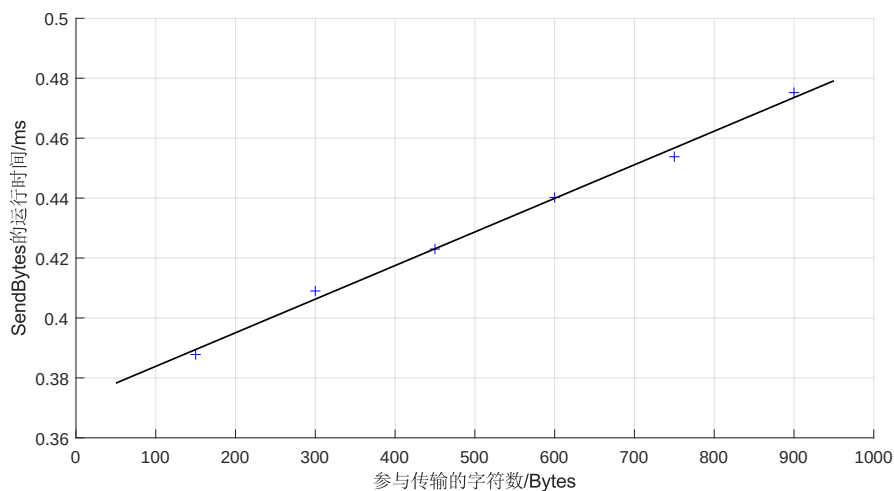
假如用一条线性直线拟合这个传输时间，则这条直线的截距即为 *SendBytes*

传输字节数/Bytes	150	300	450	600	750	900
第一次测量/ms	0.381	0.405	0.403	0.434	0.448	0.474
第二次测量/ms	0.385	0.394	0.434	0.467	0.454	0.458
第三次测量/ms	0.391	0.386	0.435	0.425	0.470	0.462
第四次测量/ms	0.381	0.469	0.417	0.453	0.443	0.473
第五次测量/ms	0.401	0.409	0.426	0.422	0.454	0.473
平均值/ms	0.3878	0.4090	0.423	0.4402	0.4538	0.4752

表 4.2 *SendBytes* 函数运行时间数据表

的准备时间。那么软总线实际的传输时间也可以被计算出。

最小二乘法（Least Squares Method）是一种数学优化技术，用于在一组数据中找到最佳函数匹配，以便最小化误差的平方和。这种方法广泛应用于数据拟合，特别是在统计学、工程学和自然科学中。最小二乘法提供了一种找出模型参数使得模型与数据的总偏差最小的有效方式。因此本研究采取最小二乘法做出传输时间关于传输字节数的拟合直线。

图 4.2 *SendBytes* 函数传输速率拟合图

通过计算，拟合直线的表达式为：

$$T = 1.1211 \times 10^{-4} B + 0.3726 \quad (4-1)$$

式中， T 为 *SendBytes* 函数总运行时间， B 为这次运行中所传输的字节数，单位为 Bytes。 B 的系数表示字节数的增加所导致的运行时间的平均增加量。因

此，本次实验中软总线的实际传输速率为：

$$V = \frac{8}{1.211 \times 10^{-4}/10^3} = 71.3558 \text{ Mbits/s} = 8.9195 \text{ MBytes/s} \quad (4-2)$$

本研究采用 `scp` 指令给出 TCP/IP 速率的参考值。`scp` 指令在同样的拓扑结构中，传输速率稳定为 7.9 MBytes/s 。可以看出，软总线对传输信道的利用更加充分，速度也更快。

4.3 分布式推理数据并行化速度

4.3.1 hmdfs 远端文件访问效率

本研究中分布式推理部分借助了 `hmdfs` 进行文件的交换，具体过程在3.5部分已经进行了介绍。本节将对其提到的步骤（3）中从远端取出文件并进行推理的时间进行测量，并与直接在本地进行推理的时间进行对比。本节选取的实验材料，是一个包含一百条待推理文本的文本文件。并且为了与系统中的子模块的实现方式对齐，本次测试采用相同的 `CPythonAPI` 调用相同的脚本进行推理测试。

由于 `Bert` 模型已经在系统初始化时加载，因此本部分实验不考虑模型加载时间。一般来说，模型加载时间在 8s-18s 不等。

首先，如果分布式推理子模块从 `hmdfs` 的 `/merge_view` 目录中取出源文本，则完整推理将包含文件传输过程和读取过程，因此必然有速度上的损失，因此相对本地推理时间一定更长。结果如表4.3所示。

测量次第	第一次	第二次	第三次	第四次	第五次	平均值
hmdfs 文本推理时间/s	129.313	127.548	127.068	127.362	127.318	127.7218

表 4.3 hmdfs 文本推理时间数据表

其次，如果从本端文件系统中取出源待推理文本，则完整推理将不包含文件传输过程。结果如表4.4所示。

测量次第	第一次	第二次	第三次	第四次	第五次	平均值
本地文本推理时间/s	123.995	122.330	122.215	122.894	122.107	122.7081

表 4.4 本地文本推理时间数据表

可以看出，在 `hmdfs` 中进行分布式推理，一条文本的平均推理时间为 1.278s，对速度的损失仅有 4.09%，几乎可以忽略不计。因此可以认为，`hmdfs` 在文件操作上的效率较高，不会对分布式推理的效率产生明显影响。

4.3.2 分布式推理模型数据并行化效率分析

假定有 N 个相同资源的设备同时在组网中参与计算，总的计算文本量为 M ，那么按照本文提出的策略，每个节点将有 $\frac{M}{N}$ 的文本量需要处理。一次指令发送时间为 τ ，一条指令从 **hmdfs** 的推理时间为 t_h 。则按照时序推导，理想化条件下一次推理完成的总时间为：

$$T_M = N\tau + \frac{M}{N}t_h \quad (4-3)$$

假定一条指令从本地的推理时间为 t_l ，则某个节点不进行并行化的总时间为：

$$T_S = Mt_l \quad (4-4)$$

根据上述分布式推理数据并行化速度测试，实际条件下 t_h 仅略大于 t_l ，并且 τ 是一个较小的数，可以忽略不计。那么按照本文提出的策略，设 $\Delta t = t_h - t_l$ ，则一次分布式推理的时间可以总结为：

$$T_M = N\tau + \frac{M}{N}t_l + \frac{M}{N}\Delta t = \frac{M}{N}t_l + t_{trans} \quad (4-5)$$

式中， t_{trans} 因为传输指令或文件损失的时间，可以视为本文的分布式推理实际完成了将总的推理时间优化到单节点的 $\frac{1}{N}$ ，仅因为传输延迟损失了少量时间。数据并行化效率较高。

4.4 本章小结

本章节对基于鸿蒙软总线的端侧集群动态管理系统中的关键部件进行了详细的性能测试。通过设置具体的测试拓扑结构，我们模拟了实际网络环境中的条件，使用两台树莓派通过路由器以 **Ethernet** 和 **WIFI** 连接形式组成局域网，评估了系统在可能的网络不稳定条件下的表现。

首先，本章节对分布式软总线的四种传输方式进行了综合评估，特别关注于字节传输函数 *SendBytes* 的性能。通过精确的时间测量方法，采用 *gettimeofday* 函数记录了不同长度字符串的传输时间，并通过统计分析得出了传输时间与字节数的关系。实验结果显示，传输时间与传输字节数之间存在线性关系，且传输效率高于常规的 **TCP/IP** 协议。

其次，本章对分布式推理任务中的数据并行化速度进行了测试。通过 **hmdfs** 系统进行文件交换和处理，对比了从远端取文件和本地取文件进行推理的时间差异。结果表明，尽管通过 **hmdfs** 进行数据交换引入了额外的时间开销，但这种开

销相对于整体推理时间是较小的，表明系统在文件操作效率上表现优异，对分布式推理的影响可以忽略不计。

最后，通过理论分析和实验数据支持，本章证明了基于鸿蒙软总线的端侧集群动态管理系统在分布式环境中具有良好的传输效率和推理性能。这些研究成果不仅展示了系统在实际应用中的高效性和稳定性，也为系统的进一步优化和应用扩展提供了坚实的基础。

第五章 总结与展望

本研究主要围绕鸿蒙生态中的分布式软总线技术展开，通过深入探讨其在以树莓派为基础的端侧集群动态管理中的设计实现和应用，实现了一套高效的系统原型。系统通过有效利用分布式软总线和基于此的分布式文件系统（hmdfs），提升了端侧设备在分布式环境中的通信与数据处理效率，同时展示了在分布式推理任务中的应用潜力。

本研究成功构建了基于鸿蒙软总线的端侧集群动态管理系统，实现了端侧设备的有效管理与调度。系统利用软总线技术优化了数据传输与处理流程，通过精确的任务调度和资源管理，提升了任务处理的效率和响应速度。

本研究对分布式软总线的通信效率进行了系统的测试和评估，结果显示软总线在数据传输方面具有较高的效率，与传统的 TCP/IP 协议相比，表现出更优的传输速度和资源利用率。此外，通过对分布式推理过程的优化，验证了基于 hmdfs 进行的数据并行化推理的可行性和效率。

本研究还展示了系统在分布式 BERT 模型推理任务中的应用，通过实际的性能测试，证明了系统在处理复杂计算任务时的高效性和稳定性，尤其是在数据并行化方面的表现。

计算机技术国产化是一个艰辛的过程，本研究在研究软总线时，经历了许多困难。分布式软总线技术刚刚起步，缺乏完备的文献支撑，也缺乏关于分布式软总线详尽的技术文档。另外，也缺乏基于软总线的消息队列服务，让分布式计算的实现缺乏异步性。在具体的实践中，也存在当前 OpenHarmony 和 OpenEuler 支持的硬件平台不够广泛，未能体现这两个操作系统灵活的可移植性。但是这些国产化技术现在已经有了独具特色的技术实践，运行效率也非常使人满意。本研究最大的意义，就是证明了分布式软总线具备作为一个分布式计算框架基底通信技术的能力。相信在未来国产开发者的继续研究下，分布式软总线将会有更加完备的服务和更广泛的应用场景，

致谢

首先，我要感谢我的指导老师杨鹏飞老师，感谢杨老师接纳我成为他的研究生。杨老师认真负责，从毕业设计的选题到最终的成果，都给出了有建设性的建议，并且定期督促我们完成毕业设计。我还要感谢郑天洋师兄，在我的进度遇到困难时，给了我许多可行的解决方案，并且帮助我完成了论文定稿，给予了我莫大的帮助。

同时，我也要感谢我的父母和姐姐，感谢你们多年以来对我的倾心支持，我才能完成我的本科学习生活。感谢时刻陪伴我，一起玩闹的兄弟们。

最后，由于学术水平有限，所写论文难免有不足之处，恳请各位老师和同学提出批评和指正。

参考文献

- [1] 王林超, “基于 harmonyos 的混合 app 中间件研究与实现,” Master’s thesis, 西南科技大学, 2024.
- [2] 刘丹雨, “基于鸿蒙操作系统的高压燃气流量积算仪研制,” Master’s thesis, 电子科技大学, 2023.
- [3] 孙冰, “500 亿鸿蒙生态圈设备数超 1.5 亿, 智能座舱最快年底上市,” 中国经济周刊, no. 20, pp. 66–67, 2021.
- [4] 叶剑, “面向鸿蒙应用的跨设备隐私泄露方法及检测方法研究,” Master’s thesis, 电子科技大学, 2024.
- [5] 吴韶鸿, “大数据开源技术发展研究,” 现代电信科技, vol. 44, no. 08, pp. 17–22, 2014.
- [6] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, “Apache spark: a unified engine for big data processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [7] 夏靖波, 韦泽鲲, 付凯, and 陈珍, “云计算中 hadoop 技术研究与应用综述,” 计算机科学, vol. 43, no. 11, pp. 6–11+48, 2016.
- [8] 董春涛, 李文婷, 沈晴霓, and 吴中海, “Hadoop yarn 大数据计算框架及其资源调度机制研究,” 信息通信技术, vol. 9, no. 01, pp. 77–84, 2015.
- [9] D. Radhika and M. Duraipandian, “Load balancing in cloud computing using support vector machine and optimized dynamic task scheduling,” in *2021 9th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO)*, Noida, India, 2021, pp. 1–6.
- [10] W. Qu, X. Ding, K. Yang, Y. Bao, and W. Chen, “Icta: Intelligent computing task allocation for efficient deep learning in distributed edge computing system of iot,” in *Proceedings of the 2020 2nd International Conference on Big-data Service and Intelligent Computation (BDSIC ’20)*. New York, NY, USA: Association for Computing Machinery, 2021, pp. 61–69.
- [11] H. Geng, D. Zeng, and Y. Li, “Performance efficient layer-aware dnn inference task scheduling in gpu cluster,” in *GLOBECOM 2022 - 2022 IEEE Global Communications Conference*, Rio de Janeiro, Brazil, 2022, pp. 2242–2247.
- [12] H. Gholami and R. Zakerian, “A list-based heuristic algorithm for static task scheduling in heterogeneous distributed computing systems,” in *2020 6th International Conference on Web Research (ICWR)*, Tehran, Iran, 2020, pp. 21–26.
- [13] W. Renbiao, Z. Zhenchi, J. Yunfei, and Q. Han, “Adaptive scheduling strategy based on deadline under cloud platform,” *Journal of Computer Applications*, vol. 43, no. 1, pp. 176–184, 2023.

- [14] P. Li, H. Seferoglu, and E. Koyuncu, “Model-distributed inference in multi-source edge networks,” in *2023 IEEE International Conference on Acoustics, Speech, and Signal Processing Workshops (ICASSPW)*, Rhodes Island, Greece, 2023, pp. 1–5.
- [15] F. PING, X. TANG, Y. PAN, and Z. LI, “Scheduling strategy of irregular tasks on graphics processing unit cluster,” *Journal of Computer Applications*, vol. 41, no. 11, pp. 3295–3301, 2021.
- [16] Z. Chen, X. Zhao, C. Zhi, and J. Yin, “Deepboot: Dynamic scheduling system for training and inference deep learning tasks in gpu cluster,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 9, pp. 2553–2567, 2023.
- [17] 程宇, 滕滨, and 董少杰, “自主可控操作系统的探索与研究,” *通信世界*, no. 09, pp. 40–41, 2021.
- [18] 倪光南, 胡浩, 宋志平, and 于德翔, “持续构筑科技、经济相融合的发展生态,” *科技导报*, vol. 39, no. 16, pp. 35–38, 2021.
- [19] 黄靖茹, 李钊兴, 裴喜龙, and 徐志宇, “用于开源学习社区的 openeuler 性能分布式测试实验平台开发,” *软件导刊*, vol. 21, no. 01, pp. 222–225, 2022.
- [20] 朱宇航, 陈勤思, 徐一凯, and 胡松, “基于国产操作系统的海湾污染物扩散输运预报系统——以象山港为例,” *海洋预报*, vol. 39, no. 05, pp. 48–59, 2022.